

When Can an HPC Agent Act?

Benchmarking LLM Reliability Across Pre-Submission Workflow Stages

Anonymous Authors
Anonymous Institution
Anonymous Email

Abstract—Large language model (LLM) agents promise to reduce the manual effort required to prepare and operate scientific workflows on leadership-class systems. Yet an HPC workflow is not a single language task: selecting an application, constructing scientifically valid inputs, choosing resources, and authoring a scheduler script have different failure modes and different operational consequences. We present the Trinity HPC Agent Benchmark, an evidence-backed evaluation of four open-weight models on these four pre-submission stages across ten application–system anchors per stage. The corpus contains 320 judged model answers (four models, two prompt arms, and ten anchors per stage), graded by a versioned requirement library with three judge replicates per answer. Across all models, software selection, resource selection, and batch-job creation pass in 60/80, 64/80, and 58/80 cases, respectively. Input preparation passes in only 16/80 cases; no evaluated model exceeds 6/20. Manual inspection shows that the latter failures are often plausible but non-runnable artifacts, including inconsistent atom counts, mismatched biomolecular sequences, and invented configuration schemas. A paired prompt-enrichment study does not establish a reliable improvement for input preparation. We therefore argue for capability-aware orchestration: use inexpensive models for bounded planning tasks behind deterministic guards, but require executable validation and/or expert review before submitting model-authored scientific inputs. The result is deliberately scoped: the benchmark is DEV-tuned, uses an LLM judge, and does not yet establish end-to-end execution success. We release the rubric, prompts, answers, verdict evidence, and report artifacts to make those limitations inspectable.

Index Terms—LLM agents, high-performance computing, scientific workflows, benchmarking, reliable AI

I. INTRODUCTION

Scientific workflows increasingly span heterogeneous supercomputers, schedulers, application stacks, data services, and domain-specific file formats. The work needed to turn a scientific intent into a submitted job is often routine but high consequence: a malformed input deck can consume a scarce allocation while yielding scientifically invalid output. LLM agents offer a compelling interface to this work. Agent designs such as ReAct interleave planning with actions and observations [1]; workflow systems such as Parsl already provide a substrate for executing distributed tasks [2]. The open question is not only whether an agent can use tools, but *which parts of an HPC workflow it can safely perform*.

We study this question in the context of *Trinity*, a multi-tenant agentic HPC workspace built at the Argonne Leadership Computing Facility (ALCF). Trinity couples Claude Agent

SDK-based reasoning and tool use with Globus identity, data-movement, and compute services; from one chat workspace, a scientist can plan work, submit and monitor jobs, move data, and receive results across authorized facilities [3]. Its facility-aware reasoning is grounded in the Application Catalog, which records systems, software installations and launch information, and performance data across ALCF, NERSC, and OLCF resources [4]. This paper evaluates the pre-submission reasoning layer; it does *not* claim an end-to-end deployment result on those facilities.

General agent benchmarks assess interactive reasoning and tool use [5], [6], but they do not test the coupled facility and scientific-format constraints that make HPC preparation risky. We introduce the Trinity HPC Agent Benchmark, which decomposes job preparation into four independently scored subtasks: software selection, input preparation, resource selection, and batch-job creation. This decomposition makes performance actionable: an orchestrator can route a bounded subtask to a lower-cost model, escalate it to a stronger model or human, or require a validator before taking an irreversible action.

Our contributions are:

- A benchmark design for four pre-submission HPC workflow subtasks, grounded in application–system anchors, versioned requirements, and answer-level evidence.
- An evaluation of four open-weight models across 320 judged answers that identifies input preparation as a qualitatively different and unsafe failure regime.
- A capability-aware deployment policy for Trinity and a transparent account of measurement limits, including rubric iteration, judge variance, and a null prompt-enrichment result.

II. BACKGROUND AND RELATED WORK

A. LLM agents and scientific workflows

LLM agents couple natural-language reasoning with external actions, observations, and feedback [1]. In scientific computing, the natural counterpart is a workflow engine: Parsl, for example, expresses and executes parallel tasks across diverse execution providers [2]. Recent work has demonstrated LLM-controlled tool calls on Polaris for molecular-dynamics workflows [7]. These systems motivate a control loop in which

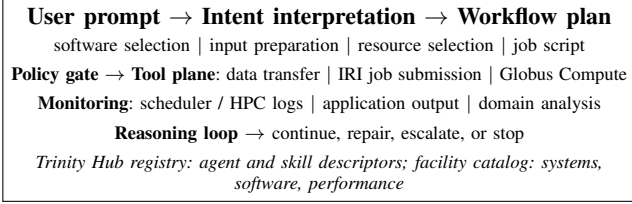


Fig. 1. Example Trinity workflow, based on the documented workspace [3]. The benchmark evaluates the four pre-submission outputs in the second row; tool use, monitoring, and iterative reasoning remain end-to-end system stages.

an agent interprets intent, selects an executable workflow, invokes tools, inspects outcomes, and replans.

Trinity operationalizes this separation in a shared workspace. Its documented primitives are *agents* (domain specialists that can plan and run campaigns), curated parameterized *workflows*, and reusable *skills* such as procedures, tool wrappers, or known fixes [3]. An embedded coding agent accesses ALCF/NERSC/OLCF integrated research-infrastructure APIs, Globus transfer and compute, and experiment tracking. Each action yields knowledge, traces, and provenance, permitting a later agent or user to inspect what happened and why. A system prompt or a tool call should not be treated as a substitute for a scientific validator.

B. Agent benchmarks and the HPC gap

AgentBench evaluates agents in multiple interactive environments [5]; GAIA emphasizes practical, multi-step assistant tasks involving tools and information retrieval [6]; and SWE-bench evaluates issue resolution in real repositories [8]. These benchmarks expose important agent capabilities, but their pass criteria do not encode scheduler syntax, site policy, application-specific input grammar, or physical consistency. An HPC benchmark must distinguish an answer that is rejected loudly by a scheduler from an answer that launches but silently models the wrong system.

III. TRINITY SYSTEM DESIGN

Figure 1 presents the documented Trinity workspace. A scientist collaborates with embedded or specialist agents in a room; workflows and skills provide reusable operational procedures; and the tool plane invokes authorized services. The benchmark focuses on four pre-submission artifacts that precede tool invocation. Recorded knowledge, traces, and provenance support inspection, validation, escalation, and recovery.

A. Skills without weight updates

The benchmark also informs how Trinity should use frontier and open-weight models. A frozen model cannot acquire a frontier model’s capability in its parameters without training. It can, however, receive procedural instructions, few-shot demonstrations, retrieved knowledge, tools, external memory, or inference-time critique. This matches Trinity’s documented skill abstraction: a reusable procedure, tool wrapper, or workflow that an agent loads, rather than a remotely callable

agent [3]. For high-risk tasks, a frontier model can serve as a critic or selector over lower-cost candidate outputs; for bounded tasks, deterministic checks can reject violations directly. This paper evaluates a limited version of context-level transfer (prompt enrichment), not active teacher–student supervision; claims about the latter require a separate experiment.

B. Example execution loop

A user may ask Trinity to run a scientific workload in plain language. First, an agent interprets the request into an intent and selects a workflow. The workflow constructs the pre-submission artifacts: an application choice, input files, a resource request, and a batch-job script. After policy validation, tool adapters transfer data and submit the job through integrated research-infrastructure (IRI) APIs and Globus Compute. Monitoring agents then inspect scheduler and HPC logs, application output, and domain-specific results. The reasoning layer uses this evidence to decide whether to continue to the next workflow step, repair an artifact, escalate to a specialist or user, or stop. The benchmark measures only the highlighted pre-submission decision point; an execution campaign must additionally measure authorization, tool reliability, scheduler acceptance, runtime behavior, and scientific validity.

C. Facility-aware knowledge plane

The Application Catalog is Trinity’s structured knowledge plane for the pre-submission stages. It separates three kinds of fact that agents otherwise tend to conflate in generated prose: (i) a system description (facility, hardware, queues, and endpoints), (ii) an application description (installed module, build path, and launch command), and (iii) measured performance records for an application–system pair [4]. This separation supports a typed workflow: software selection queries availability; resource selection queries queue limits and scaling data; and batch generation consumes a verified launch recipe. Input preparation still requires application- and domain-specific contracts beyond the catalog, which motivates its distinct evaluation and stronger policy gate.

D. Registry, runtime, and trust boundary

The public Trinity Hub is a registry, not the runtime: it stores public descriptors of agents and skills, while a contributing agent continues to run on its owner’s infrastructure [3]. A Trinity operator registers an agent endpoint with an out-of-band credential; during a room turn, the runtime sends the conversation transcript to that endpoint and receives a response. This boundary is material to an HPC agent design: remote agents must treat transcripts as untrusted input, credentials must remain outside the public registry, and scientific-workflow policy gates must apply before external effects. The benchmark’s evidence-first design complements these operational controls by recording why an artifact passed or failed before it reaches the tool plane.

IV. BENCHMARK DESIGN

A. Tasks and corpus

An *anchor* is a (subtask, application, system) triple. We use ten anchors per subtask across PBS Pro and Slurm schedulers and across molecular dynamics, quantum chemistry, CFD, LLM inference, and benchmark workloads. The four subtasks represent the pre-submission chain:

- 1) **Software selection**: choose an appropriate installed application for a described workload.
- 2) **Input preparation**: author the required, runnable input artifacts without inventing binary/runtime files.
- 3) **Resource selection**: choose queue, nodes, ranks, and walltime consistent with site constraints and scaling guidance.
- 4) **Batch-job creation**: write a scheduler script that requests legal resources and invokes reachable inputs and the selected application.

Four open-weight models—Nemotron-3-Ultra, Gemma-4-31B, GPT-OSS-120B, and Llama-3.1-8B—answer each anchor at temperature 1.0. Each anchor has base and enriched prompt arms, yielding 320 answers (4 models $\times$ 4 tasks $\times$ 10 anchors $\times$ 2 arms). The rich arm mechanically inserts additional facility context where appropriate; it is paired with the base arm to avoid confounding from regenerated outputs.

B. Requirements and grading

Prompts state the task but withhold the answer under test. They are generated from a vendored, byte-verified copy of the Application Catalog at commit 4c6a192242452d798afb6b7f1e44362f263a1c7d. A reference answer gives the judge one valid solution, while the rubric explicitly records free-choice dimensions. Requirements compose from general, subtask, application, scheduler, and system YAML files. Each requirement records a claim, decision method, severity (fatal/major/minor), evidence source, and rationale.

The judge receives the composed requirement library and returns a verdict and quoted evidence per rule. Deterministic checks run independently. Scores are derived outside the judge: a fatal violation makes the relevant dimension zero and marks a fatal error; a major violation gives that dimension one; minor-only violations retain two. A pass requires 2/2 and no fatal error. Three judge replicates are aggregated by majority, rather than averaging scores. The corpus was graded by gpt56terra; this introduces a possible model-family preference that we do not measure.

C. Statistical scope

We report pass counts because the deployment question is binary: can this artifact clear all required gates? The paired enrichment analysis uses a Wilcoxon test on per-row violation counts. Because only ten anchors populate a model-subtask cell and 20/320 cells flip across judge replicates, differences smaller than approximately five percentage points should not be interpreted as model rankings. The TEST split remains

unopened: the reported corpus is a DEV result after 28 rubric versions. These decisions deliberately favor inspectability over a premature generalization claim.

V. RESULTS

Table I shows a separation by subtask. Across models, software selection, resource selection, and batch-job creation pass in 75%, 80%, and 72.5% of attempts, respectively. Input preparation passes in only 20%; no model exceeds 6/20. Nemotron-3-Ultra is the strongest overall at 60/80, while Llama-3.1-8B has the highest software-selection count but zero input-preparation passes. Thus, a single overall score hides the operational decision that matters: models should be assigned by failure mode, not merely by aggregate rank.

A. Input preparation fails quietly

Input preparation is not simply a lower score on the same scale. Frequent violations include physical-system value mismatches (48 rows), invented keywords (27), missing mandatory sections (24), truncation (19), and missing required files (10). Examples include a coordinate file declaring 13 atoms for a 34,000-atom system, a quantum input with `nat=64` above 16 coordinates, a haemoglobin sequence under a ubiquitin header, and an invented AlphaFold `input.json` schema. These artifacts look plausible and may bypass superficial checks, unlike an invalid queue name that fails at submission. The conclusion is therefore not that every model response is useless; it is that model-authored scientific inputs need executable and domain-aware validation before unattended use.

B. Other subtasks are more tractable but remain guarded

Software selection loses passes primarily through choosing the wrong application, concentrated in GROMACS@Sirius and LAMMPS@Polaris. Resource-selection failures are largely due to explicit queue-limit and rank-arithmetic violations. Batch-job failures include non-invoked applications and unreachable inputs. These errors are often amenable to catalog, scheduler, and static checks. They support controlled assistance and auto-repair, not unrestricted authority.

C. Prompt enrichment does not establish transfer

The enrichment A/B is negative on the available evidence. Among 30 byte-identical control anchors, passes change from 93 to 89 ($p = 0.039$). Among ten input-preparation anchors, passes change from 1 to 5 ($p = 0.21$). Enrichment reduces targeted invented-keyword violations (16 to 11), but input preparation is all-or-nothing: a remaining fatal or major violation still prevents a pass. The treated effect is smaller than the control drift, so we do not ship enrichment as a reliability intervention based on this result.

VI. DISCUSSION: CAPABILITY-AWARE TRINITY POLICIES

The benchmark leads to a practical policy table rather than a universal “agent capable/incapable” label.

For the three more tractable subtasks, Trinity can use small or open-weight models as candidate generators, retaining a

TABLE I
MAJORITY PASS COUNTS (OUT OF 20 ATTEMPTS PER MODEL-SUBTASK CELL) UNDER CORPUS V8 AND RUBRIC R28.

Model	Software	Input prep.	Resource	Batch	Total
Nemotron-3-Ultra	16	6	18	20	60/80 (75%)
Gemma-4-31B	16	6	19	16	57/80 (71%)
GPT-OSS-120B	10	4	20	10	44/80 (55%)
Llama-3.1-8B	18	0	7	12	37/80 (46%)
All models	60/80	16/80	64/80	58/80	198/320 (62%)

TABLE II
SUGGESTED TRINITY POLICY, DERIVED FROM OBSERVED FAILURE MODES.

Stage	Agent role	Required gate
Software selection	Candidate proposer	Catalog availability and workload-fit check
Resource selection	Candidate proposer	Queue-limit, rank, and allocation validators
Batch creation	Draft generator	Scheduler linting plus path/application validation
Input preparation	Assisted author only	Executable parser/run checks and domain review

deterministic policy gate before action. A frontier model is most valuable as a targeted critic or selector when a rule is semantic rather than syntactic. For input preparation, the correct default is escalation: retrieve an application-specific skill, generate candidate artifacts, run format and domain validators, and require a human or specialized verifier when validation is incomplete. This policy localizes expensive inference and human attention where benchmark evidence says they matter.

The benchmark also illustrates why skill transfer should be evaluated as a system property. Prompting and retrieval may give a frozen model instructions and knowledge; a frontier critic may improve a specific output; candidate selection can use test-time search. None of these changes the frozen model’s weights, and none should be represented as intrinsic capability without an ablation that measures it. Future experiments should compare (i) a base model, (ii) a retrieved procedural skill, (iii) a frontier critique loop, and (iv) validated multi-candidate selection under the same held-out anchors and execution checks.

VII. THREATS TO VALIDITY AND ARTIFACT PLAN

The paper’s main limitation is measurement maturity. Twenty-eight rubric versions were fit against DEV labels and the TEST split was never opened. Ten anchors per cell limit resolution; $k=3$ judge consensus still flips 20/320 cells; and judge self-preference is unmeasured. Some Slurm rules lack execution evidence because the run archive is PBS-only. Format-contract requirements are careful expert reading, not physical execution validation, and the current code-block extractor can mask malformed per-file output. A severity choice on a single PBS-directive rule changes the headline

from 198/320 to 206/320, so we report the choice rather than hide it.

The artifact package contains the rubric, registry, prompts, reference answers, all model answers, majority verdicts with quoted evidence, and a report renderer. It also records the exact Application Catalog revision used to generate prompts; the catalog itself should be retrieved from its upstream source rather than redistributed without a license determination [4]. The score matrix is derived from stored verdicts rather than copied into prose. Before submission, we will freeze the rubric, pre-register the primary endpoint, open a held-out TEST split, add execution-backed checks for Slurm and representative application inputs, and report inter-rater or cross-judge agreement.

VIII. CONCLUSION

Trinity aims to make HPC workflows agentic without treating every workflow step as equally safe. Our benchmark shows that open-weight models can often support software, resource, and batch-script planning under guardrails, but that input preparation remains unreliable in a way that is particularly dangerous: outputs can appear valid while encoding the wrong scientific system. The appropriate next step is not an unconditional autonomous agent; it is a capability-aware controller that makes validation, escalation, and evidence first-class workflow operations.

REFERENCES

- [1] S. Yao, J. Zhao, D. Yu, N. Du, I. Shafraan, K. Narasimhan, and Y. Cao, “ReAct: Synergizing reasoning and acting in language models,” in *International Conference on Learning Representations*, 2023. [Online]. Available: <https://arxiv.org/abs/2210.03629>
- [2] Y. Babuji, K. Chard, L. Gerhardt *et al.*, “Parsl: Pervasive parallel programming in python,” in *Proceedings of the 28th International Symposium on High-Performance Parallel and Distributed Computing*, 2019, pp. 25–36.
- [3] Argonne Leadership Computing Facility, “Trinity hub documentation: Multi-tenant agentic HPC workspace,” Online documentation, 2026, accessed: 2026-09-22. [Online]. Available: <https://docs.trinityscience.org/>
- [4] Application Catalog Contributors, “Application catalog: Cross-facility software catalog for ALCF, NERSC, and OLCF systems,” GitHub repository, 2026, accessed: 2026-09-22. [Online]. Available: https://github.com/zhenghh04/application_catalog
- [5] X. Liu, H. Yu, H. Zhang *et al.*, “AgentBench: Evaluating LLMs as agents,” in *International Conference on Learning Representations*, 2024. [Online]. Available: <https://arxiv.org/abs/2308.03688>
- [6] G. Mialon, C. Fourrier, C. Swift, T. Wolf, Y. LeCun, and T. Scialom, “GAIA: A benchmark for general AI assistants,” in *International Conference on Learning Representations*, 2024. [Online]. Available: <https://arxiv.org/abs/2311.12983>

- [7] H. Ma, A. Brace, C. Siebenschuh, G. Pauloski, I. Foster, and A. Ramanathan, "Connecting large language model agent to high performance computing resource," *arXiv preprint arXiv:2502.12280*, 2025. [Online]. Available: <https://arxiv.org/abs/2502.12280>
- [8] C. E. Jimenez, J. Yang, A. Wettig *et al.*, "SWE-bench: Can language models resolve real-world github issues?" in *International Conference on Learning Representations*, 2024. [Online]. Available: <https://arxiv.org/abs/2310.06770>